Sistemas operativos

Jaime Nazar Anchorena

Contents

Practica 1: POSIX **3**
- Entorno de desarrollo . . . 3
- Crear procesos . . . 3
- `ps` . . . 4
- Copy on write . . . 4
- File management . . . 4
- `pipe` . . . 6
  - dup . . . 6

Practica 2: Herramientas de trabajo **6**
- Unit testing . . . 6
  - Unit test . . . 6
  - Test driven development . . . 7
  - Testing en C . . . 7
  - CuTest . . . 7
  - Unit test structures & patterns . . . 7
  - Git hooks . . . 7
- Workflow GIT . . . 7
  - Branch . . . 8

Teorica 1: Introduccion **8**
- Sistemas operativos . . . 8
  - Administrador de recursos . . . 8
- Revision hardware . . . 9
  - Procesador . . . 9
  - Memoria . . . 9
  - Dispositivos I/O . . . 10
  - Booteo . . . 10
- Tipos de SO . . . 11
- System calls . . . 11
  - Read . . . 11
  - Waitpid . . . 11
  - Kill . . . 12
  - Fork . . . 12

A donde va la salida estandar . . . 12
Estructura de un sistema operativo . . . 12
Monolithic systems . . . 12
Microkernels . . . 12
Virtual machines . . . 13

Teorica 2: Procesos **15**
Pseudo concurrencia . . . 15
Reflexion . . . 16
Creacion de procesos . . . 17
Win32 . . . 17
Terminacion de procesos . . . 17
Jerarquia de procesos . . . 18
Grupo de procesos . . . 18
Win32 . . . 18
Estado de procesos . . . 18
Transcisiones . . . 19
Implementacion de procesos . . . 19
Modelado multiprogramacion . . . 20

Teorica 3: Inter process communication(IPC) **20**
Memoria compartida . . . 20
Pasaje de mensajes . . . 22
Sincronizacion . . . 22
Buffering . . . 22
Ejemplos de IPC . . . 22
Files . . . 22
Pipes . . . 23
Files vs. pipes . . . 23
Sockets . . . 23
FIFO vs Unix Domain Sockets . . . 23
Signals . . . 23
Race condition . . . 24
Region critica . . . 24
Mecanismos de sincronizacion . . . 24
Deadlock . . . 24

Teorica 4: Threads **24**

Practica 1: POSIX

Entorno de desarrollo

Trucos de bash:

- `ctrl + r` para buscar historial
- `!!` expande con el ultimo comando (`$sudo !!`)
- `file.{jpg, png}` expande a `file.jpg file.png`
- `$(...)` da el output de un comando (`touch file-$(date -I)`)
- `ctrl + a` para ir al comienzo de la linea
- `ctrl + e` para ir al final de la linea
- `cd -` vuelve al dir que estabas antes
- `fc` fix command, abre el ultimo comando en un editor y corre la version editada
- `type` indica de si algo es builtin, program, o alias (`type ping`)

En bin/ estan todos los comandos de administracion, para configurar el sistema. Por otro lado, en bin/ estan los comandos de uso general.

Crear procesos

Se pueden crear utilizando `fork`, syscall que la llamo una vez y vuelve dos veces, el proceso original y el que se creo.

Los dos procesos son identicos salvo el valor de retorno de `fork`. Si `fork` no puede crear un proceso(raro, pero puede pasar), devuelve -1.

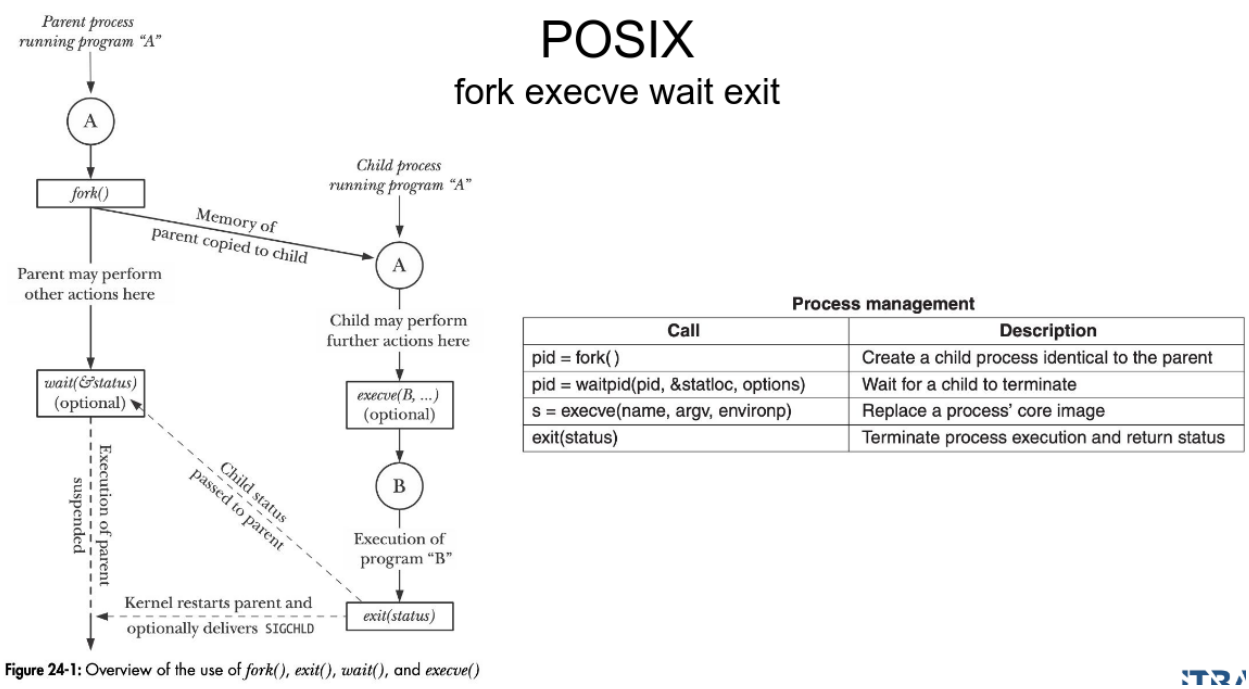

Figure 1: Pipeline fork

Como cada proceso puede crear varios procesos, pero tiene un solo padre, si quiero saber cual es el `pid` del

padre es necesario que el padre sepa cual es el `pid` del hijo. Esto se logra devolviendo el `pid` del hijo en el padre.

Es muy importante la llamada a `wait` porque sino queda un monton de procesos **undefined**, consume entradas en la tabla de procesos. Todas las variables que tenga un proceso original, incluyendo los punteros, el hijo los hereda. Es importante por ese motivo cerrar todos los **file descriptor**.

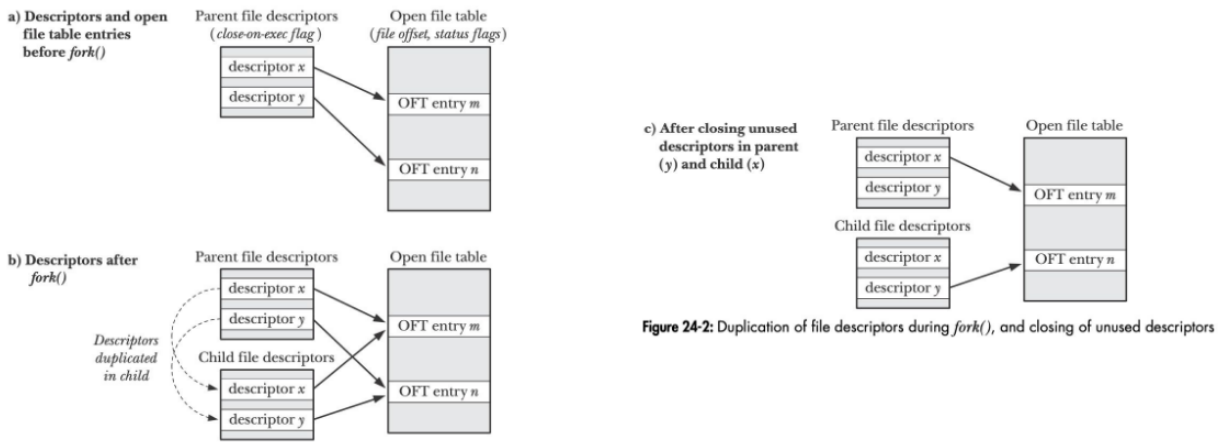

Figure 2: File descriptors fork

`ps`

Muy recomendado que lo aprendamos a usar bien para debuggear los TP. Se corre generalmente con `aux` pued indica: a+x -> mostrar todos los procesos, u -> incluir la columna con informacion del usuario de cada programa. Se puede combinar con otros arg(todo junto):

- Muestra los procesos que estan corriendo en el momento. Usando `w` como argumento(wide) muestra los argumentos que se utilizaron para correr cada programa.

- Si se corre con `e` va a mostrar las variables de entorno.
- Si se corre con `f` se mostrara un arbol de los procesos con caracteres ASCII

  Nota: Hay mas arg en la presentacion de campus.

Copy on write

Los procesos clones usan la misma memoria que el padre(heap, stack, memory maps), copiar esa memoria cada vez que usamos fork es lento y ocupa memoria. Entonces, linux permite que ambos procesos utilicen la misma memoria fisica y solo copia la memoria cuando alguno de los procesos intenta escribir. Linux implementa esta dandole la misma **page table** a cada programa, pero marcadas como **read only**. Entonces, cuando alguno intenta escribir ocurre una **page fault**.

File management

Todo lo que es I/O es universal en Unix, se maneja como si fuera un archivo. Es muy eficiente el read write, lo unico que cambia entre tipos como lo accedo. Para trabajar con archivos como humanos les asignamos

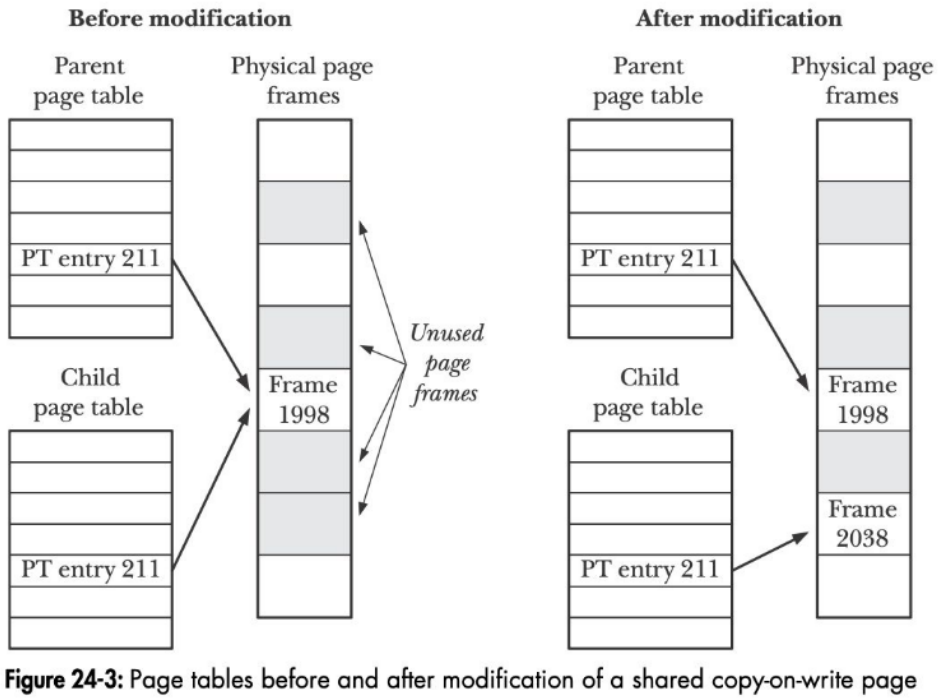

Figure 3: Representacion de copy on write

nombres, pero las computadoras prefieren tratar con numeros. Por lo tanto, cuando trabajamos con bajo nivel utilizamos **file descriptors** los cuales indican la referencia a un archivo **abierto**.

Una vez con el fd se puede leer y escribir, el fd 0 es la salida estandar. Comandos utiles:

- `head`, primeras lineas de un archivo
- `tail`, ultimas lineas de un archivo
- `stat`, muestra metadata del archivo
- `lseek`, corre el puntero a otra posicion

Una vez termino de trabajar con un fd, se libera y el numero queda disponible para cuando abra algun otro archivo. Cuando abro un archivo, usa el fd mas chico disponible. Cuando se lee de disco es muy importante que lo que uno lea o escriba este alineado con el **tamano del bloque** para evitar fragmentacion, no es obligatorio, pero es una buena practica.

Un archivo tiene un formato predefinido? No, para el sistema operativo no importa, hay metadata guardada en disco para saber su ubicacion o permisos, pero el archivo en si son bytes para el SO. Es problema de la aplicacion saber que formato tiene, si tubieran formatos para el SO seria demasiado complejo e innecesario.

> **Nota**: Cada proceso tiene su tabla con el mapeo de fd a nombre, es un error muy comun tener una tabla global para todos los procesos(esta mal eso). Todo lo que comentan es **local** al proceso. Si dos procesos tienen abierto el mismo archivo el fd no tienen porque coicidir.

`pipe`

Es un canal donde se envia todo lo que saca un proceso se lo manda a otro, es unidireccional y es un metodo de comunicacion. Es el mas sencillo que hay. Es un buffer compartido, donde un proceso escribe y otro lee, una cola. No hay posibilidad de reabrir un pipe, pues es un objeto anonimo(no tiene nombre).

En caso de que el programa que lee termine y el otro siga escribiendo se lanzara una senal de error nivel sistema operativo. No es comportamiento normal, por lo tanto se lanza un error. Detecta que el otro programa dejo de leer pues desaparece el fd. Pero que pasa si deja de escribir el programa, que pasara con el que lee? Se vacia el pipe y cuando en ultimo read devuelva 0, entonces se cierra el pipe.

Otro caso, el proceso que lee, lee despacio, entonces, se llena el buffer. Cuando la otra parte quiera escribir y el SO ve que el buffer esta lleno, bloquea la parte que escribe hasta que se pueda ejecutar write. No da error, pues esto es una situacion normal.

dup

El manejo del pipe se realiza de la siguiente manera:

1. Corro pipe en shell. Se llama syscall `pipe`
2. `pipe` devuelve a la shell fd escritura y fd lectura
3. Se hace un fork, de manera que los hijos(las dos shell) tienen los mismo fd que el padre(pues se hacen copias identicas).
4. La shell cierra los dos fd, pero siguen abiertos para los hijos. Se separa del pipe.
5. Se elimina el fd de lectura de un hijo y el de escritura para el otro. Quedando el diagrama de pipe convencional.
6. Luego para el proceso 1 se configura el stdout como el pipe y para el proceso 2 es el stdin. Duplicando los fd, cerrando stdout y stdin para los proceso.
7. Una vez duplicados a std y stdin, se cierran los originales.
8. Se hace el exec en las shell.

Aca child 1 y child 2 son una shell, al final del todo se hace el exec y se ejecutan los programas. Esta es una de las razones por la cual stderr esta separado de stdout.

> **Nota**: Para ver la representacion grafica ver apuntes. Puede haber dos fd, uno duplicado, abiertos, pero es mala practica.

Practica 2: Herramientas de trabajo

Unit testing

No nos lo van a pedir, pero sirve un monton. Puede haber un bug en el codigo, pero tambien puede haber un bug en el codigo que prueba el codigo. Para evitar esta situacion hay varias tecnicas.

Unit test

Se utilizan para probar por separado todas las partes de forma independiente, si se cambia el codigo entonces se pueden detectar errores.

> Todo junto se escribe separado.

Sin embargo, puede ocurrir que se pasen todos los test y al integrar no funcione nada. De manera que hay que prestar especial atencion a la integracion.

Test driven development

Se utilizo en otros cuatrimestras, pero fue descartado. Basicamente se comienza con los test y luego se va al codigo.

Testing en C

Hay varias maneras:

- Usar la macro assert, no recomendado ya que aborta con el primer fallo y no da informes.
- Hacer tu propia libreria de testeos
- Usar frameworks de terceros

CuTest

Framework para realizar testeos, conviene usar este ya que es muy simple y lo suficientemente completo para el alcance de la materia. Corre todos los test, no aborta, y presenta mensajes mas detallados. Ademas, tiene mayor variedad de funciones, entonces mejora la calidad.

Terminos

- Test unitarios: Funciones que reciben por parametro un `CuTest *` e invocan alguna funcion de la libreria.
- Suite de test: Funcion que retorne un `CoSuite *` y tiene al menos un test unitaro. Engloba los test que pertencen a un modulo en particular.

  Nota: Tener un archivo separado para cada suite.

Unit test structures & patterns

- Behavior-Driven Development(BDD), Given-When-Then(Precondiciones-Determidado caso-Entonces)

Repo de materia con ejemplo: https://github.com/alejoaquili/c-unit-testing-example

Git hooks

Son scripts que Git ejecuta antes/despues de algun evento. Estan en `.git/hooks`, eventos

- Pre-commit
- Post-commit
- Pre-push
- etc.

Si falla, corta el commit.

Workflow GIT

Workflow basico de git para poder trabajar de manera ordenada y minimizar errores. La idea es tener un sistema de tracking de cambios de un set de archivos,

Branch

Representa un espacio de trabajo que contiene el codigo del repositorio y sobre la cual se pueden generar distintas versiones.

A su vez git tiene un buen sistema para unificar los cambios realizados en varias branches si se tiene una jerarquia correcta entre distintas ramas. Generalmente se tiene la ultima version funcional en main, mientras que en la branch develop se tiene lo ultimo de desarrollo(es la branch mas activa). Luego, se tienen las branch de develop que son las features, entonces se hace marge de una feature con develop primero y por ultimo de hace un merge con main. Comandos utiles:

- rebase
- stash
- cherry pick
- log
- reset

Teorica 1: Introduccion

Sistemas operativos

Primero planteemos que ocurriria si no hubiese un sistema operativo, entonces, `printf` no funciona, no tiene sentido. No hay drivers, no hay nada definido y no existe el SO que puede proveer el servicio de imprimir en pantalla.

En conclusion, el sistema operativo provee servicios(resuelve problemas) a nivel de llamar una funcion que haga algo, abstrayendo los detalles(no me importa en que hardware lo este corriendo).

Provee un conjunto obstracto y limpio de los recursos, administra estos recursos. Se requiere que el hardware tenga por lo menos dos modos de operacion(con distintos privilegios) para que funcione correctamente.

Oculta el hardware y ofrece una interfaz limpia, elegante y consistente al programador, las abtracciones son una de las claves para comprender SOs.

Administrador de recursos

Que pasa si 2 o mas procesos intentan imprimir en la impresora?

El SO es un ente centralizado que agrega cada impresion a un queue y garantiza que el acceso al recurso impresora sea correcto.

Administrar recursos incluye **multiplexar** estos recursos, mediante alguno de los siguientes metodos:

- Tiempo
- Espacio

Multiplexar trata de la distribucion de un recurso(temporal o espacialmente), por ejemplo, dividir el auditorio de SDT en clases mas chicas(espacial).

Revision hardware

Procesador

Un registro importante es el RFLAGS.

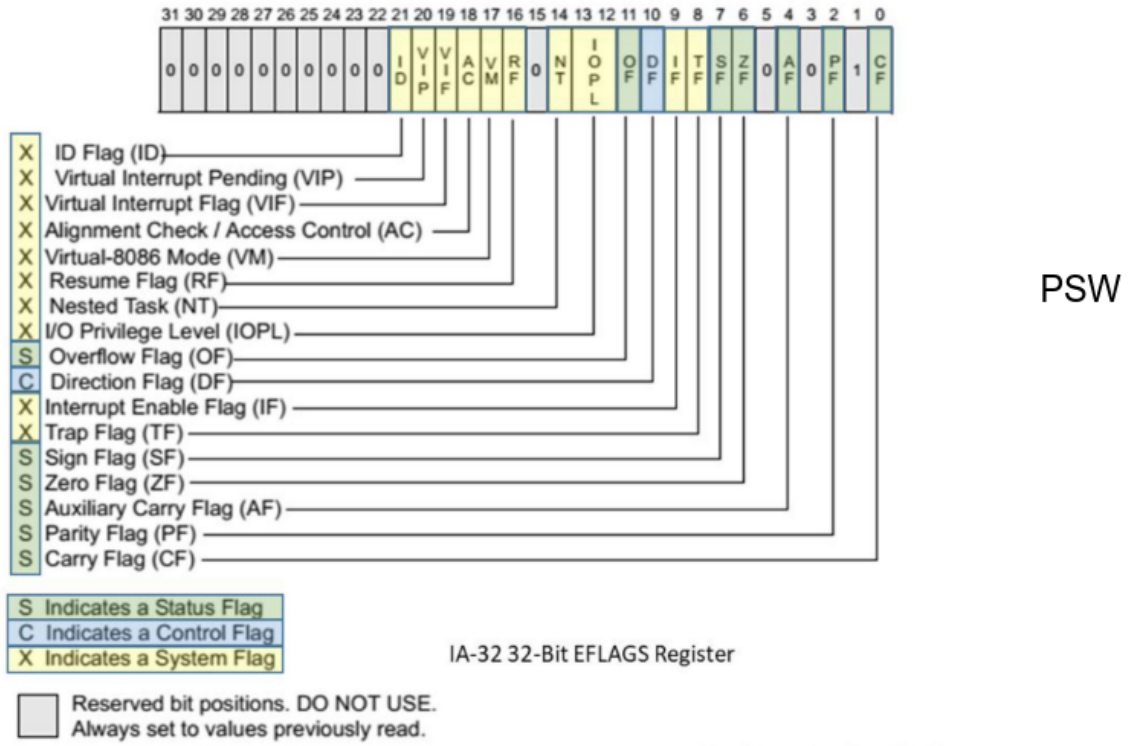

Figure 4: RFLAGS

Al iniciar el hardware, el stack estara vacio y el IP apuntara a la direccion de nuestro programa, pero primero hay que determinar cual es esa direccion.

Multithreading Mantener el estado de 2 threads e intercambiar entre ellos rápidamente cuando sea necesario. No es paralelismo.

Multicore Replicar los núcleos independientes. Pueden soportar múltiples threads.

> **Nota**: Ley de Moore es una aproximacion bastante acertada, pero llegamos a una limitacion de temperatura. Hoy en dia simplemente se replican los nucleos(muchos nucleos en un mismo procesador).

Memoria

Idealmente la memoria debería ser: - Extremadamente rápida (más que la ejecución de una instrucción) - Absurdamente grande (para albergar a todos los procesos juntos) - Asquerosamente barata

Obviamente esto es una utopia porque en la practica no se cumplen estas condiciones, por lo tanto, surgue una jerarquia de memoria(ver figura).

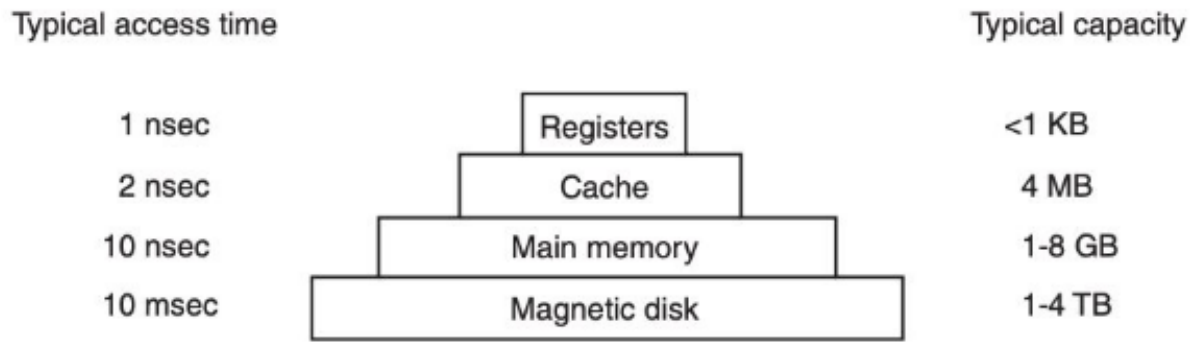

Figure 5: Jerarquia de los tipos de memoria

Dispositivos I/O

Un dispositivo consta de 2 partes:

- Controlador (abstracción)
- Dispositivo

Hay 3 formas de IO:

- Busy waiting(polling), en este caso, el cpu pregunta(es un derperdicio)
- Interrupción
- DMA (Direct Memory Access)

Proceso de interrupcion(?):

1. El driver le indica al controlador qué hacer. Este inicia la operación en el dispositivo.
2. El controlador señaliza al controlador de interrupciones (CI).
3. Cuando el CI está listo (otras interrupciones) setea el pin correspondiente en el CPU.
4. El CI escribe el número de dispositivo (vector de interrupciones) en el bus.

Booteo

- La placa madre posee un programa llamado BIOS (Basic I/O System)
- Flash RAM, no volátil pero actualizable por el SO
- La BIOS posee instrucciones para interactuar con la pantalla, teclado y disco

Proceso de booteo(una vez se determina cual es el dispositivo de booteo:

1. Chequear cuánta memoria tenemos y si ciertos dispositivos como el teclado están instalados y responden correctamente.
2. Escanea buses PCIe y PCI en busca de dispositivos instalados.
3. Determina el dispositivo de booteo.
4. Se carga el primer sector en memoria y se ejecuta
5. Este sector suele leer una tabla de particiones y se se carga un segundo sector
6. Se carga en memoria el SO y se ejecuta.

7. El SO consulta a la BIOS los dispositivos conectado

Tipos de SO

Ordenados de mas grandes a mas chicos.

- **Mainframe**: Mucho almacenamiento y memoria. Orientado a muchos procesos. Soporta trabajos por lotes, transacciones y timesharing. OS/390, tendencia a UNIX.
- **Server**: Computadoras personales o incluso mainframes. Impresoras, archivos o webs. Solaris, FreeBSD, Linux y Windows Server.
- **Multiproceso**: (Necesaria la separación)
- **Personal Computer**: Proveer soporte a un único usuario. Linux, FreeBSD, Windows y Apple's OS X.
- **HandHeld Computer**: Tablets y smartphones. Android y iOS
- **Embedded**: Electrodomésticos y teléfonos antiguos. ROM, no permite la instalación de aplicaciones. Embedded Linux, QNX y VxWorks.
- **Sensor-node**: Redes de nodos. Poseen CPU RAM y ROM. Sensan diferentes condiciones. Conexión inalámbrica. TinyOS
- **Real Time**: Necesitamos que una acción ocurra en un momento específico. hard vs soft. Industria, aviónica, milicia y multimedia. eCos y FreeRTOS
- **Smart Card**: Pagos electrónicos. Del tamaño de una tarjeta de crédito. Restricciones de potencia y memoria muy altas. Obtienen energía por inducción o al conectarse a un lector.

System calls

Como vimos en arqui, las syscall buscan:

- Proveer un conjunto abstracto y limpio de los recursos (hardware) -> entender qué hace el SO
- Administrar estos recursos -> transparente para el proceso
- Se puede ver como una simple llamada a función que además cambia a modo kernel
- El mecanismo para realizar una system call depende de la arquitectura y debe expresarse en assembler.

POSIX es basicamente la API de linux, en windows se llama **win32**, nosotros vamos a ver el grupo principal de syscalls definidos por POSIX.

> **Nota**: Malloc puede derivar de una syscall, pero en realidad solo es necesario realizar una syscall cuando se acaba la memoria disponible para pedirle mas memoria al kernel.

Read

Para verificar que la primera vez que se ejecuto `read` levanto todos los datos posibles, se corre una segunda vez, si devuelve 0 significa que no hay mas nada para leer (si le pedis 1024 y te da 1000 y despues 0, entonces el archivo es de 1000 bytes) y la lectura fue exitosa. Sin embargo, si la segunda vez devuelve mas que 0(en el caso de leer un archivo) entonces no se leyo todo y hay que volver a hacerlo.

Waitpid

Hace que el proceso espere a que termine el proceso con el pid especificado. Durante la espera el programa se queda bloqueado y no ocupa ciclos del procesador, pero si memoria. Se entera el parent porque el SO termina la syscall.

Por otro lado, el SO se entera de que termino por la llamada a `exit()`(la cual es una syscall) del child. Se fija quien es el parent y si el mismo esta esperando lo notifica.

> **Nota**: Hay mas syscall en la T1.

Kill

Manda una aenal que alerta a otro proceso, simplemente le avisa, pero no necesariamiente lo mata. Ahora, si envio la senal -9 ahi si termina. Cada programa tiene un handler para las senales, entonces los handlers se general en runtime y se setean con syscalls. Esto permite que un programa en vez de terminar muestre una ventana que pregunte al usuario si esta seguro de que quiere terminar el proceso.

El SO hara que el IP cambie a apuntar la funcion del handler, lo hara sin importar si la funcion no esta en su lugar y la va a ir a buscar. Implementa **zero trust**, se va a asegurar que el programa termine si se envia la senal de terminacion.

Fork

Mis palabras:

> Se crea un proceso identico que sigue la ejecucion en el momento que retorna.

Asi es, **KISS**. Es un ejemplo de las varias funciones no siguen lo usual(exec(no retorna), exit(no retorna), etc.): llamo una vez y retornan una vez.

A donde va la salida estandar

Ejecutan write en fd 1, no les importa a donde apunta el fd, eso depende del SO. Algunos ejemplos:

- `ls` -> terminal,
- `ls | grep` -> pipe
- `ls > foo.txt` -> archivo
- `ls > /dev/null` -> dispositivo

Estructura de un sistema operativo

Monolithic systems

Toda la funcionalidad del SO esta en un solo binario, toda funcion tiene visibilidad del resto de las funciones. Esto es eficiente, pero complejo. Un error en cualquier funcion hace fallar el sistema operativo completo. Ademas, soporta **shared libraries** o **DDLs**.

Presenta una filosofia de estructura, ver figura. El TP arqui es monolitico(como el TPE de SO), el user space esta separado, pero no es parte del SO.

> **Nota**: Es muy usado, por ejemplo, Windoes, FreeBSD, Linux, etc.

Microkernels

La idea es dejar los componentes esenciales en el kernel(lo minimo), mientras que el resto se va a correr como clientes del kernel(en user-mode). Los drivers corren como un proceso mas, por lo que un error fuera

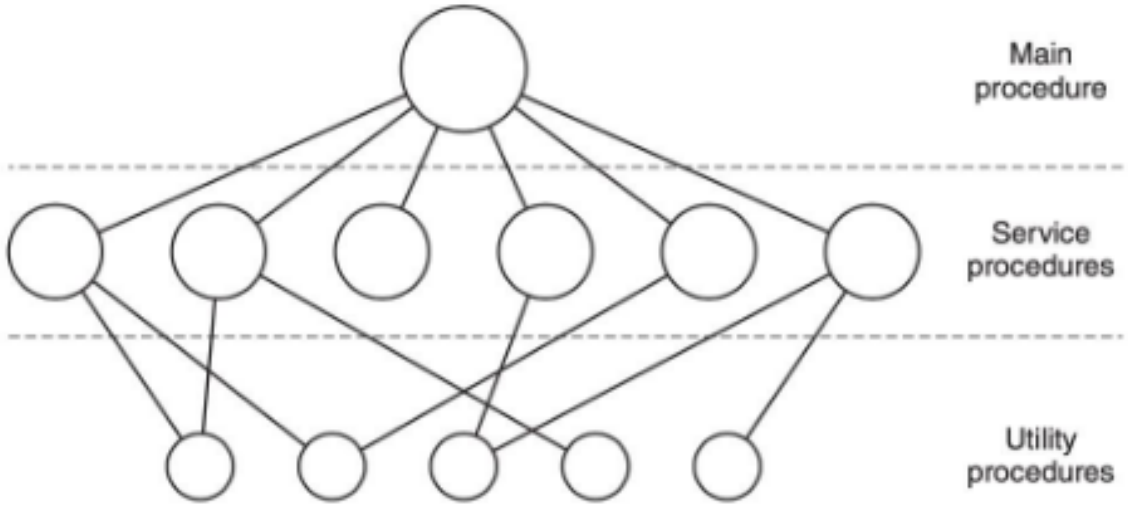

Figure 6: SO monolitico

del kernel no hace fallar al SO. Tiene alta confiabilidad(Real time), implementado por Integrity, PikeOS, MINIX, etc.

Un ejemplo, MINIX, presenta un **reincarnation server** el cual es el padre de todos los drivers y servers, chequea constantemente de controles que esten todos vivos. Limpia los drivers y servers muertos, y chequea una tabla para ver que hacer(generalmente los reinicia).

Mecanismo vs politica La separacion entre mecanismo y politica es una buena estrategia de programacion pues permite reducir el tamano del kernel y separar areas diferentes. Colocar el mecanismo en el kernel y la política por fuera, por ejemplo, asignar prioridades a los procesos(por fuera del kernel) y ejecutarlos según estas prioridades:

- Elegir al proceso de mayor prioridad (mecanismo) puede estar en el kernel
- Asignar las prioridades (política) puede estar por fuera del kernel.

Nota: Hay implementaciones hibridas que combinan ambas.

Virtual machines

Tradicionalmente Mail server, FTP server, Web server, etc corren en computadoras separadas, lo cual lo hace popular para Web hosting: servidor dedicado vs. compartido - costo y flexibilidad Tambien presentan utilidad para uso personal. Para que funcione, el CPU debe ser virtualizable: SO corriendo en user mode ejecuta instrucción privilegiada, es necesario que haga un TRAP al virtualizador para emularla en software. Pentium ignoraba estas instrucciones aunque existían intérpretes como Bochs (bajo rendimiento).

No es exactamente lo mismo que un emulador de hardware, pero si hace creer a los SO dentro de los VM que son su propia maquina con su propio CPU, RAM, etc. Son maquinas separadas, lo cual lo hace conveniente

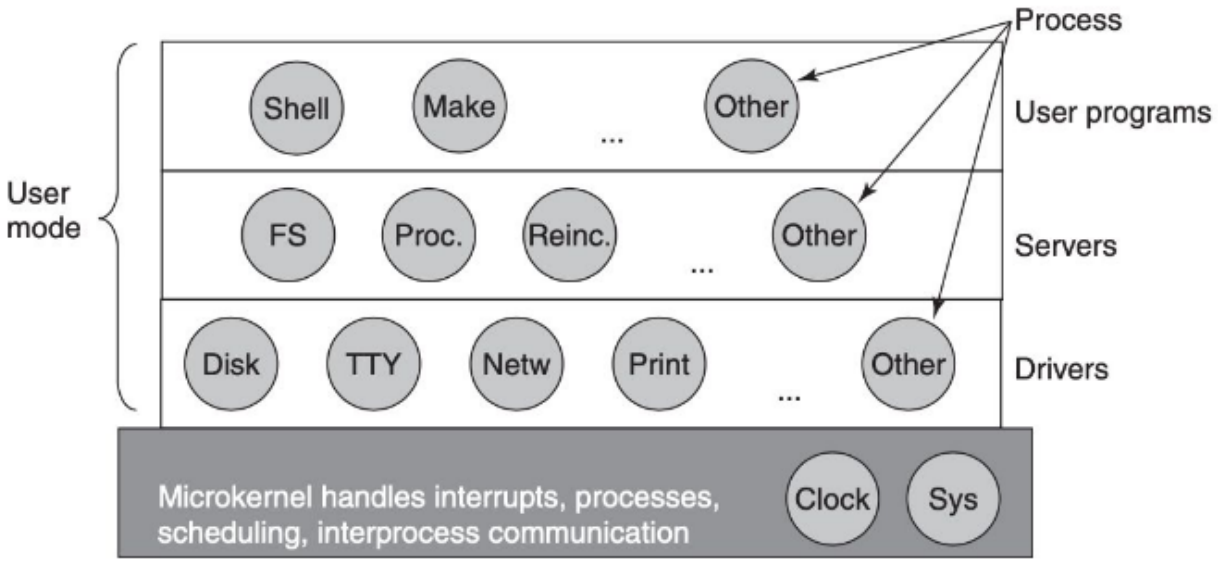

Figure 7: SO microkernels

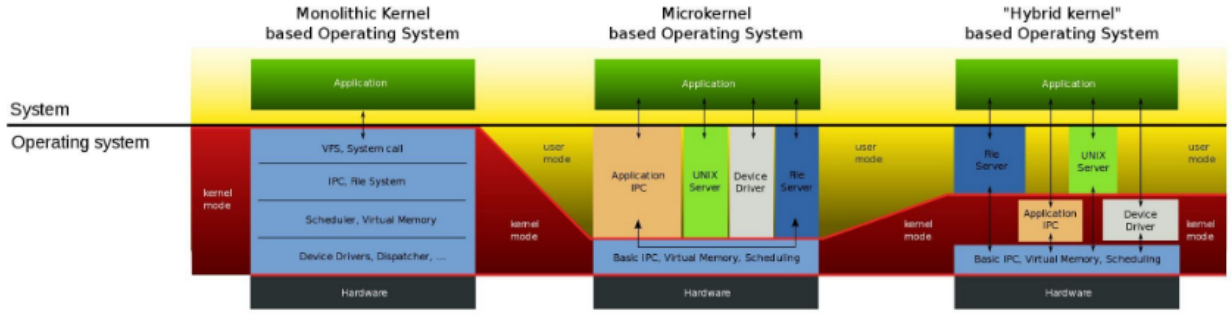

Figure 8: Monolitico vs Microkernels

por cuestiones de costo y seguridad.

Hay diferentes tipos de maquinas virtuales, tambien conocidos como **hypervisors**:

- **Virtual machine monitor o type 1 hypervisor**: Se ocupa de proveer los servicios necesarios.
- **Type 2 hypervisor**: Tiene un SO por debajo que se encarga de abstraer los procesos, file system, etc. QEMU y BOCHS.
- Sistemas híbridos.

Es mas eficiente y rapido el tipo 1 debido a que tiene menos capas e interactua directamente con el SO. El tipo 2 tiene ya una capa extra.

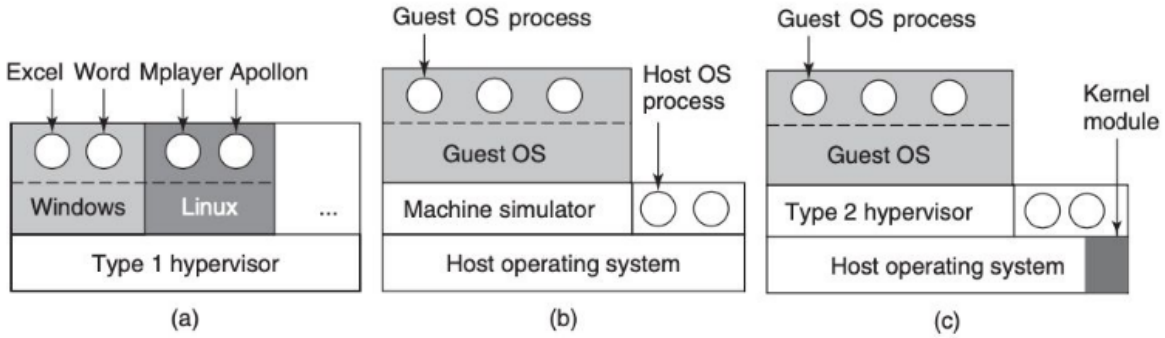

Figure 9: Tipos de VMs

Teorica 2: Procesos

Todo software se organiza en **procesos secuenciales** o simplemente **procesos**, hay que diferenciar bien entre:

- **Proceso**: Abstraccion de programa en ejecucion (si ponemos esto en el examen va a estar bien).
- **Programa**: Almacenado en disco, no hace nada.

  Nota: Esto puede ser pregunta de examen, como todo.

Que podria ocurrir si tenemos un programa que corre dos veces? Lo que ocurre es que tengo dos procesos y un programa.

Para que un programa pueda ser ejecutado, se debe cargar el codigo en memoria para poder convertirse en proceso. Posee sus propios registros y variables, habra un stack por cada proceso.

> Desde el punto de vista del proceso, el es dueno del CPU.

Esto se debe a que el proceso no se tiene que preocupar por los otros procesos, por ejemplo, saber si el CPU esta ocupado o si otro lo quiere usar. Por esto se llama proceso secuencial, para el proceso es una instruccion atras de la otra, no percibe los bloqueos de tiempo.

Pseudo concurrencia

Periodo de tiempo muy corto, llamado **quantum** o **time slice**, en el cual se corre un proceso antes de ser interrupido. Estos periodos son tan cortos que son imperceptibles para el usuario.

Tambien se lo suele llamar **pseudo paralelismo**, no es concurrencia porque en realidad no esta ocurriendo todo al mismo tiempo. Para tener paralelismo real se necesitan de multiples nucleos compartiendo memoria fisica, eso es **paralelismo real**.

No hay que pensar en los **switches**, es decir, que no notamos los saltos entre procesos dentro del CPU. Este switch no es uniforme ni reproducible ya que no puedo hacer supuestos sobre tiempo de ejecucion.

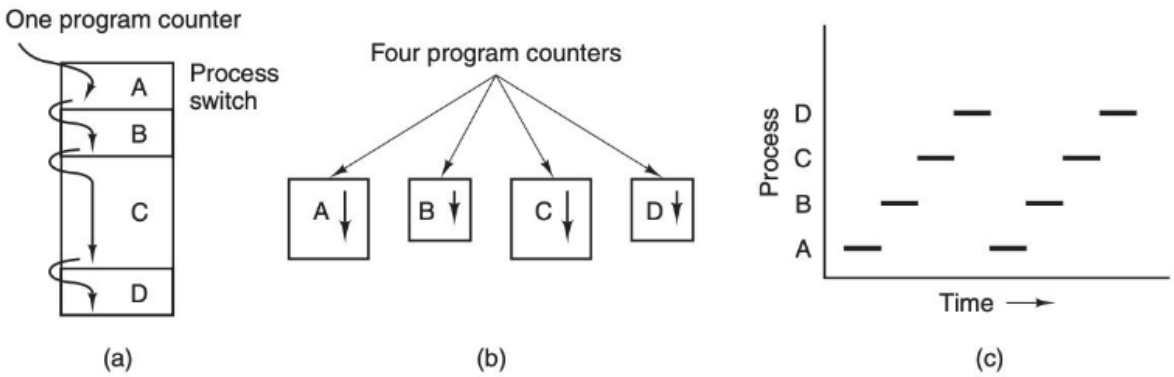

Figure 2-1. (a) Multiprogramming four programs. (b) Conceptual model of four independent, sequential processes. (c) Only one program is active at once.

Figure 10: Modelo de proceso

Reflexion

Cuando se tiene un bug, es deseable que sea facilmente replicable, que sea aislable. Luego, si se tiene dos programas con memoria compartida, donde el proceso 0 es el que initializa la variable compartida y el 1 la utiliza, por los switches, no puedo suponer que 0 se ejecutara antes de 1.

Este es un supuesto de tiempo de ejecucion que no es correcto hacer, entonces, a veces funciona y a veces no, dependiendo de como se ejecuto el switch, haciendo que sea dificil de replicar. La idea es programar de manera que seamos inmunes al orden de ejecucion del SO.

> **Nota**: El context switching es costoso para el CPU.

Ademas, la responsabilidad principal del **scheduler** es repartir de manera homogenea el tiempo de ejecucion dentro del CPU de cada proceso, la manera mas simple es haciendo una lista circular(0123012301230123...) e ir ejecutando cada programa por una cantidad de tiempo determinada antes de ir al siguiente(se conoce como **round rolling**).

> Siempre conviene hacer algo funcional(basico) primero(no tiene que ser eficiente ni lindo) para poder empezar con algo, para asegurar que es correcto y usar de referencia. Permite hacer experimentos y verificar que todo funcione todo primero.

Creacion de procesos

Un proceso tiene una vida en el SO: nace -> ejecuta -> muere. La creacion de un proceso requiere de un SO, cualquier cosa que no este siendo creada por un kernel no es un proceso, por ejemplo, una syscall. Puede tener el mismo codigo y hacer lo mismo, pero no sera un proceso.

Casos que requieren de un proceso:

- Se inicia un SO(Daemons).
- Proceso solicita la creacion de otro(Script bash y comandos UNIX).
- Usuario solicita crear un nuevo proceso(Terminal, GUI).
- Inicio de un trabajo por lotes.

Siempre es un proceso creando otro proceso, entonces, de donde sale el primer proceso?

El sistema operativo es la respuesta correcta, no el "proceso" sistema operativo, pues el SO provee la abstraccion de un proceso, pero el por si mismo no es un proceso. De igual manera que el scheduler no es un proceso. La creacion del primer proceso es mas "artesanal", es la excepcio a la regla, luego los demas van a ser creados como se describio antes.

Win32

Para crear un proceso se llama a `createProcess()` el cual tiene 10 parametros:

- El programa a ejecutar
- Parámetros
- Argumentos
- Atributos de seguridad y control
- Prioridad
- Ventana a asociar con el proceso

Hay mas de 100 syscalls extras para administrar y sincronizar procesos, en ambos casos(UNIX y Win32) los procesos creados tienen su propio espacio de direcciones, diferente al padre.

UNIX es mucho mas elegante ya que tiene mucho menos syscalls y no cambiaron los prototipos de las funciones. Basicamente, el **core** se ha mantenido igual durante un monton de tiempo.

Terminacion de procesos

Un proceso puede terminar:

- Salida normal(voluntaria)
- Salida por error(voluntaria)
- Error fatal(involuntario), se envia una senal y el comportamiento por defecto es terminar(no se puede cambiar el manejador).
- Muerto por otro proceso(involuntario), por senal, pero si se puede cambiar manejador.

  Nota: Exit es una syscall. Si necesito una syscall para iniciarlo, entonces necesito una syscall para terminarlo.

Jerarquia de procesos

Un proceso y su creador preservan una relacion, generalmente padre / hijo. No significa que un proceso sea mas importante que otro, sino que se trata de una relacion entre procesos. En UNIX, el proceso padre de todos los procesos es `init`, es la raiz del arbol. Si se mata a un proceso padre, se manda una senal al padre y muere, pero si quiero matar a los hijos voy a tener que mandar una senal a cada uno, no mueren en cascada.

> **Nota**: En nuestros TP02, la shell va a ser nuestro `init`, aunque podriamos usar otro proceso, suele ser la shell.

Grupo de procesos

Un padre y sus hijos se denominan un grupo, esto tiene ventajas como poder mandar una senal a un grupo entero o atencion de senales independiente. En general, grupo de procesos que esta leyendo del teclado tiene un solo miembro. Para `init`, se puede secir que es analogo a la clase `object` en Java.

Win32

No existe jerarquia de procesos, el padre recibe un handler para manejar al proceso. El handle se puede transferir a otro proceso.

Estado de procesos

Un proceso puede tener uno de 3 estados fundamentales:

- Running(usando el CPU en ese instante)
- Ready(ejecutable; detenido para dejar que se ejecute otro proceso)
- Blocked(no se puede ejecutar hasta que ocurra algun evento externo)

En general, los primeros dos podrian no implementarse, se podria dejar simplemente como `runnable` y listo, no es siempre necesario llevar registro de los otros dos. Blocked, por otro lado, si debe tenerse en cuenta.

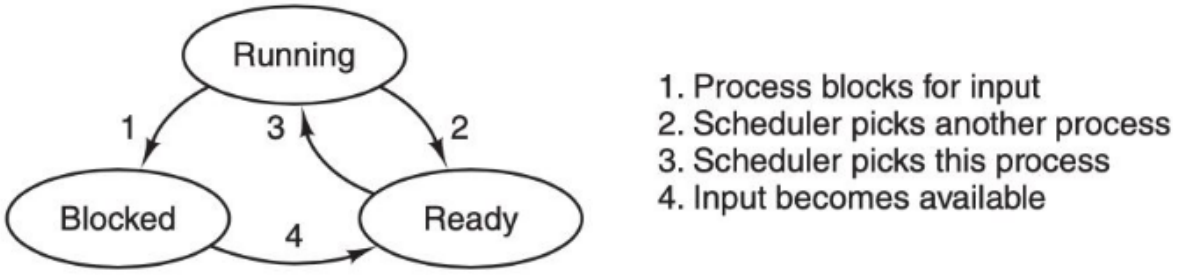

Figure 2-2. A process can be in running, blocked, or ready state. Transitions between these states are as shown.

Figure 11: Ciclo de vida de un proceso

Que un proceso este bloqueado(syscall implicita) significa que no se le da tiempo en el CPU hasta que ocurra la condicion esperada, esto se ve cuando se espera el input de un fd pues se usa la syscall `read()`.

> **Nota**: Se puede verificar el estado de un programa con `strace`.

Transcisiones

Clasico ejemplo de **ejercicio de final**:

- **Evento que pasa de running a blocked**: waitpid
- **Evento que pasa de blocked a Ready**: exit
- **Evento que pasa de running a ready**: interrupt del timer
- **Evento que pasa de ready a running**: interrupt del timer(saca a un proceso y agarra uno en ready y lo pone en running)

Pregunta de parcial: Se rompió el timer. No existe otra fuente de int. ¿puede el SO volver a tomar el control?

Si, ya que estan las interrupciones de software, las syscall. La ejecucion de una syscall no necesariamente le devuelve el control al proceso. Una vez le damos el control al kernel, no tenemos garantia de lo que va a pasar.

Pregunta de parcial: Porque no hay transicion de ready a blocked y de blocked a running?

La transicion de running a blockeado ocurre de manera sincronica, mientras estando ready no puedo pasar a blocked simplemente porque no se esta ejecutando, no tendria ningun sentido la transicion. No se puede realizar division por 0 o inv. opcode estando ready.

> Para desbloquear el proceso tiene que primero ser desbloqueado para ready por el so luego de algo externo, entonces no pasa nunca de blocked a running sin un intermedio en el que se pone en ready mientras corre algo más.

Implementacion de procesos

El SO mantiene una tabla de procesos donde almacena informacion privada de cada proceso. Es una tabla de structs, el struct pueden ser tan complejo como uno quiera. Cada entrada se denomina Process Control Block(PCB). Es la informacin necesaria para pasar de ready -> running -> blocked.

Si el proceso termino, a quien le puede interesar el valor de retorno de dicho proceso?

Al padre, al SO no le va a importar. Lo unico que le importa al SO es si el proceso termino. El padre puede conocer el exit status de su hijo mediante el PCB, pero no directamente pues es del SO. Entonces, se usa una syscall en particular, se podria usar la syscall `wait()`. Recien cuando hace `wait()` el padre, el SO borra el PCB.

Entonces, si el hijo termino, pero no se borro el PCB, se coloca el estado **zombie**. Esta en ese estado desde que ejecuta `exit()` hasta que alguien ejecute `wait()`. Si muere el padre, los hijos quedan en estado **huerfano** y son adoptados por `init`, de manera que rapidamente dejan de ser huerfanos, es decir, esta el estado, pero dura muy poco. Entonces, para evitar que se acumulen los huerfanos, `init` hace `wait()` a todos estos procesos.

> **Nota**: En el TP02 los registros se guardan en el stack(el mismo del proceso que estoy frenando).

Cuando se interrumpe un programa, se crea un nuevo stack para el scheduler/interrupt handler que no sea accesible por el programa que acabo de ejecutar. No es buen diseno que sea el mismo stack que el programa anterior pues ese stack sera usado por funciones del kernel y eso no es bueno por cuestiones de seguridad(uno con el stack puede ver un monton de datos, como valores de variables locales, etc.).

Porque necesita uno por proceso el kernel?

Porque hay funciones que por ahi no retornaron de un proceso, no haria falta un stack por proceso si se hace todo y se popea todo del stack antes de volver al programa. Sin embargo, las funciones que bloquean como`wait()` no retornan inmediatamente y todas esas funciones tienen su direccion de retorno en el stack del kernel, por eso hay un stack del kernel por proceso.

Modelado multiprogramacion

Caso:

- Un proceso usa el CPU el 20% del tiempo, el resto esta esperando por I/O(mientras corre).
- Entonces 5 procesos mantendrian el CPU al 100%.

Realista?

NO. Sera alrededor del 70%, por ese motivo la multiprogramacion es muy efectiva. Permite aprovechar el hardware tanto como se pueda.

Teorica 3: Inter process communication(IPC)

Los procesos cooperativos se distribuyen una tarea entre todos, mientras que los independientes hacen todo ellos.

Motivacion:

- Compartir informacion
- Acelerar computacion
- Modularidad
- Conveniencia

Hay dos modelos fundamentales:

- Memoria compartida
- Pasaje de mensajes

Memoria compartida

Los procesos acuerdan compartir memoria fisica(para ello hay que hablar con el SO), generalmente no requiere de syscalls para acceder a la informacion. La organizacion de la informacion y sincronizacion es determinada por los procesos. Surgen problemas de cache coherency al aumental los nucleos.

Meter al kernel en el medio de la memoria compartida no tiene ningun sentido, no hace falta acceder a la memoria con un fd, es un gasto de syscalls. Lo que hay que hacer es usar el puntero a la direccion de la memoria.

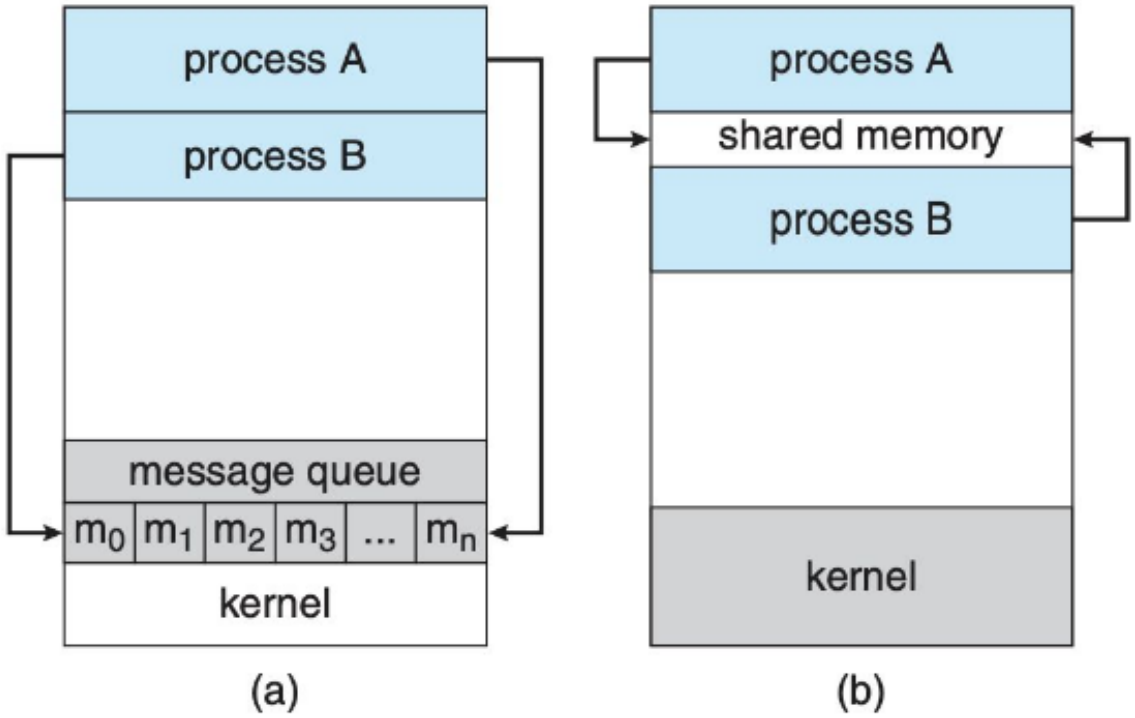

Figure 3.12 Communications models. (a) Message passing. (b) Shared memory.

Figure 12: Dos modelos fundamentales

Para que dos procesos accedan a la misma porcion de memoria se utiliza el nombre, el cual es una ruta en el filesystem (no es absoluta, en el manual aparece cual es la absoluta).

Hay que tener cuidado al manejar los punteros de la memoria compartida, pues no es el mismo para diferentes procesos por mas que sea la misma porcion de memoria. Por lo tanto, para posiciones relativas conviene leer el offset y compartir ese dato entre procesos, no el puntero en si.

Pasaje de mensajes

En este caso el kernel interviene, impone una sincronizacion implicita, no hace falta hacer polling sino que se bloquea el proceso. Es más simple en sistemas distribuidos, util para pequeñas cantidades de información y tiene un tamaño de mensaje fijo o variable.

Operaciones:

- send(message)
- receive(message)

Es necesario un canal de comunicación (memoria, bus, red)

Sincronizacion

`send` y `recieve` pueden ser bloqueantes o no bloquearse. Hay varias opciones:

- **send bloqueante**: hasta que llega a la casilla / proceso.
- **send no bloqueante**: resume inmediatamente.
- **receive bloqueante**: hasta que hay mensaje disponible.
- **receive no bloqueante**: recibe mensaje o null.

Buffering

Los mensajes residen en en un buffer

- **Capacidad 0**: no buffering -> send debe bloquear.
- **Capacidad acotada**: send debe bloquear si se agota el espacio.
- **Capacidad no acotada**: send no bloquea.

  No es conveniente usar syscalls en los programas que se bloquean.

Va a haber que desglozar el programa en dos modulos distintos si tiene que hacer cosas mientras espera a la informacion(no bloqueante).

Ejemplos de IPC

Files

Un archivo es una forma de comunicar procesos que se adapta bastante bien al modelo de memoria compartida, en este caso si interviene el kernel, pero el kernel no indica como almacenar las cosas.

Generalmente no se lo menciona como memoria compartida, pero para Ariel si se puede interpretar de ese modo.

Pipes

Mecanismo muy "estrecho" de pasaje de mensajes, solo en un sentido. Fue uno de los primeros mecanismos de IPC en UNIX, muy simple y muy usado. Tiene un buffering acotado y la sincronizacion se basa en `send` bloqueando al llenarse el pipe, mientras que `recieve` bloquea hasta que haya algo.

Hay dos formas de definir pipes:

- **Ordinarios/Anonimos**: No tienen nombre, significa que no tienen una entrada en el file system.
- **Named pipes**: Tienen nombre, permite comunicar dos procesos ordinarios.

Si no tiene un path(anonimo) como puede el otro programa encontrar el pipe?

Basicamente, se hereda, pues se mantienen los fd al hacer `fork()`. Pero no se limita solo a padre-hijo, sino que en UNIX se podria acceder a los fd correspondientes leyendo del directorio `/proc`.

Files vs. pipes

Ventajas de los pipes:

- Se eliminan automáticamente (anónimos)
- Cantidad arbitraria de información. Si el pipe se llena se bloquea el que envia, en cambio un archivo tiene un limite de espacio.
- Ejecución paralela
- Sincronización implícita

Pipe tambien es mas rapido, pues en caso de que haya un disco(podria no haberlo en UNIX) es muy lenta la escritura a disco. Se puede lograr practicamente lo mismo, pero al final del dia es mucho mas conveniente usar pipe.

Sockets

Permite comunicar procesos a traves de una red, es bidireccional(full duplex). Un socket se identifica con una IP y un puerto, generalmente sirve para una aplicacion cliente - servidor.

FIFO vs Unix Domain Sockets

Los pipes que usamos son simples, tienen una sola direccion. Los sockets se pueden usar por muchos procesos a la vez, tienen un hilo escuchando conexiones entrantes y apenas llega una comunicacion se crea un socket paralelo para la comunicacion, pero eso no se muestra, desde afuera parece un solo socket(transparente para el cliente).

Como la radio de un puerto, todos van al canal 16, pero luego prefectura los manda a otro canal y libera el 16, la unica diferencia es que hay un canal individual para cada embarcacion en caso de sockets.

> **Nota**: Las signals son async, ya que el programa que recibe una signal no esta esperando la signal, sino que esta corriendo y es interrumpido.

Signals

Basicamente como kill es una syscall, el SO se encarga de buscar el proceso al que se le envio la signal y en base a la entry en el SO del programa se llama al handler. Pero el handler no lo llama el SO, sino que

modifica el stack y los punteros del programa receptor de la signal para que la proxima instruccion que ejecute sea la del handler(el handler se corre en el contexto del programa).

No hay signals anidadas, pero tampoco es que no se recibe la signal. Lo que ocurre es que se recibe la signal y se espera a que termine el handler, una vez termina el SO procesa la otra signal. No hay un concepto re `ret` en un handler, el programa se interrumpte y luego de terminar el handler continua la ejecucion, pero no tiene porque hacer algo. Se podria definir una variable que se modifique en el handler para notificar al programa que este se ejecuto, pero nada mas.

Race condition

Una situación en las que dos o más procesos están leyendo o escribiendo datos compartidos y el resultado final depende de quién corre en cada momento.

No es sinonimo de estado inconsistente, sino que el resultado depende de la ejecucion. Es muy feo debuggear programas donde ocurren condiciones de carrera.

Esto se soluciona con una especie de patron de sincronizacion, es decir, el programa que escribe tiene acceso exclusivo y una vez termina libera el acceso exclusivo. Esto no impide que el escritor sea interrumpido, pero si hay garantia que ningun otro proceso podra escribir al mismo lugar. No es inmune al **context switch**.

Se ejecuta de manera atomica uno y despues se ejecuta otro.

Region critica

Es una parte de un programa, la parte que accede a la memoria compartida.

Mecanismos de sincronizacion

- **Semaforos**: El wakeup waiting bit se plantea(Dijkstra - 1965) como entero representando la cantidad de wakeups disponibles, el cual llamaremos semaforos. Tiene dos operaciones: **down** y **sleep**.

  Nota: Mutex es basicamente **mutual exclusion**.

Si dos quieren acceder al mismo semaforo, el primero que llega "gana el semaforo". Es una especie de race condition, pero no importa porque no afecta el resultado del problema.

El semaforo mutex es solo para proteger el acceso a la zona critica, mientras que el otro semaforo simplemente bloquea alguno de los dos procesos. Ver problemas clasicos de los apuntes.

Deadlock

Un conjunto de procesos está bloqueado (en estado de deadlock) si cada proceso del conjunto está esperando un evento que solo puede causar otro proceso del conjunto.

Teorica 4: Threads

Tradicionalmente un proceso tiene su espacio de direcciones y un nuico hilo(thread) de ejecucion.

Existen casos en los que es muy util tener multiplas hilos en un mismo espacio de direcciones corriendo de forma pseudo paralela como si fueran procesos separados, salvo por el espacio de direcciones.

Un proceso solo ejecuta una instruccion en particular en cualquier momento, no confundir esto con la nocion de threads. Todos los hilos tienen el mismo codigo, heap y variables globales. En cuanto al stack, no pueden tener el mismo, es privado a cada hilo.

La motivacion es poder correr muchas actividades simultaneas, algunas bloqueantes. Desglosar la situacion:

- Aumenta el uso del CPU
- Simplifica la programacion
- Aprovecha arquitecturas con multiples CPUs

Tiene la misma nocion de modelo de procesos, donde se pensaba en procesos secuenciales, pero con el agregado de que comparten un espacio de direcciones.

Ademas, son mas 'baratos' pues se crean y se destruyen, y son hasta 10-100 veces mas rapido que un proceso. Un thread tambien se puede reutilizar, cambiando su funcion.

Un ejemplo seria el web server, un thread es el **dispatcher** que recibe conexiones entrates y delega la atencion de una conexion a otro thread, cada conexion termina con un thread individual.